

MODULE 14

Time Series Analysis

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Time Series Analysis

What is Time Series?

A time series is data collected over time at regular intervals.

Components:

Trend: long-term direction (upward, downward)

Seasonality: regular periodic patterns

Cyclical: irregular patterns over years

Noise: random variation

Applications:

Stock prices, weather, sales, IoT sensors, web traffic

Time Series with Pandas

```
import pandas as pd
import matplotlib.pyplot as plt

# Load time series
df = pd.read_csv('data.csv', parse_dates=['date'], index_col='date')
df = df.sort_index()

# Resample
df_monthly = df.resample('M').mean()
df_weekly = df.resample('W').sum()

# Rolling statistics
df['rolling_mean'] = df['value'].rolling(window=7).mean()
df['rolling_std'] = df['value'].rolling(window=7).std()

# Plot
plt.figure(figsize=(12, 4))
df[['value', 'rolling_mean']].plot()
plt.show()
```

Stationarity and Decomposition

```
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose

# ADF test for stationarity
result = adfuller(df['value'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
if result[1] < 0.05:
    print('Series is stationary')
else:
```

```
print('Series is NOT stationary - apply differencing')
df['diff'] = df['value'].diff()

# Decompose trend, seasonality, residual
decomposition = seasonal_decompose(df['value'], model='additive', period=12)
decomposition.plot()
```

ARIMA Forecasting

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

# ARIMA(p, d, q)
# p: autoregressive order
# d: differencing order
# q: moving average order
model = ARIMA(train, order=(2, 1, 2))
fit = model.fit()
forecast = fit.forecast(steps=30)

# SARIMA for seasonal data
sarima = SARIMAX(train, order=(1,1,1), seasonal_order=(1,1,1,12))
fit = sarima.fit()
forecast = fit.forecast(steps=12)
```

ML for Time Series

```
from sklearn.ensemble import GradientBoostingRegressor
import numpy as np

# Create lag features
def create_features(df, n_lags=5):
    for i in range(1, n_lags+1):
        df[f'lag_{i}'] = df['value'].shift(i)
    df['day_of_week'] = df.index.dayofweek
    df['month'] = df.index.month
    df['quarter'] = df.index.quarter
    return df.dropna()

df_features = create_features(df.copy())
X = df_features.drop('value', axis=1)
y = df_features['value']

# Train on all but last 30 days
model = GradientBoostingRegressor()
model.fit(X[:-30], y[:-30])
```